JEPA-INSPIRED LATENT ROUTING FOR DOMAIN-SPECIALIZED LoRA EXPERTS

A proof-of-concept study combining Mixture-of-Experts routing, Low-Rank Adaptation, and a Joint Embedding Predictive Architecture objective — what worked, what didn't, and why.

Elias Tadele | Research Presentation

OVERVIEW

What this presentation covers

01

The problem

Why small LLMs need specialization, not just scale

02

Three building blocks

LoRA, Mixture-of-Experts, and JEPA — explained simply

03

System architecture

How the three combine into one working pipeline

04

Results

Routing accuracy and generation quality, on real data

05

Why it underperformed

Three concrete, checkable mechanisms

06

Roadmap

What would need to change to make this usable

Small models can't be good at everything

The trade-off

A 1.5B-parameter open model is cheap to run but **can't match GPT-4-class breadth** across every technical domain.

Fine-tune narrowly → strong in one domain, weak elsewhere.

Fine-tune broadly → diluted quality everywhere.

THE IDEA THIS PROJECT TESTS

Train several small specialists instead of one generalist, and learn a smarter way to pick which specialist answers each question.

- LoRA → cheap, swappable specialists
- MoE → pick the right specialist per query
- JEPA → a candidate way to learn that picking function

Three ideas, one system

LoRA

Low-Rank Adaptation

Small, cheap "patches" that specialize a frozen base model without retraining it.

MoE

Mixture-of-Experts

Route each query to only the relevant specialist(s) instead of using all of them at once.

JEPA

Joint Embedding Predictive Architecture

A self-supervised way to learn representations by predicting meaning, not exact wording.

LoRA: patching, not retraining

Instead of updating all of a model's weights, LoRA freezes the base model and trains a small pair of low-rank matrices that get added on top.

$$W' = W_0 + \Delta W \qquad \Delta W = B \cdot A, \quad \text{rank}(A,B) = r \ll d$$

18.46M

trainable parameters per expert

1.18%

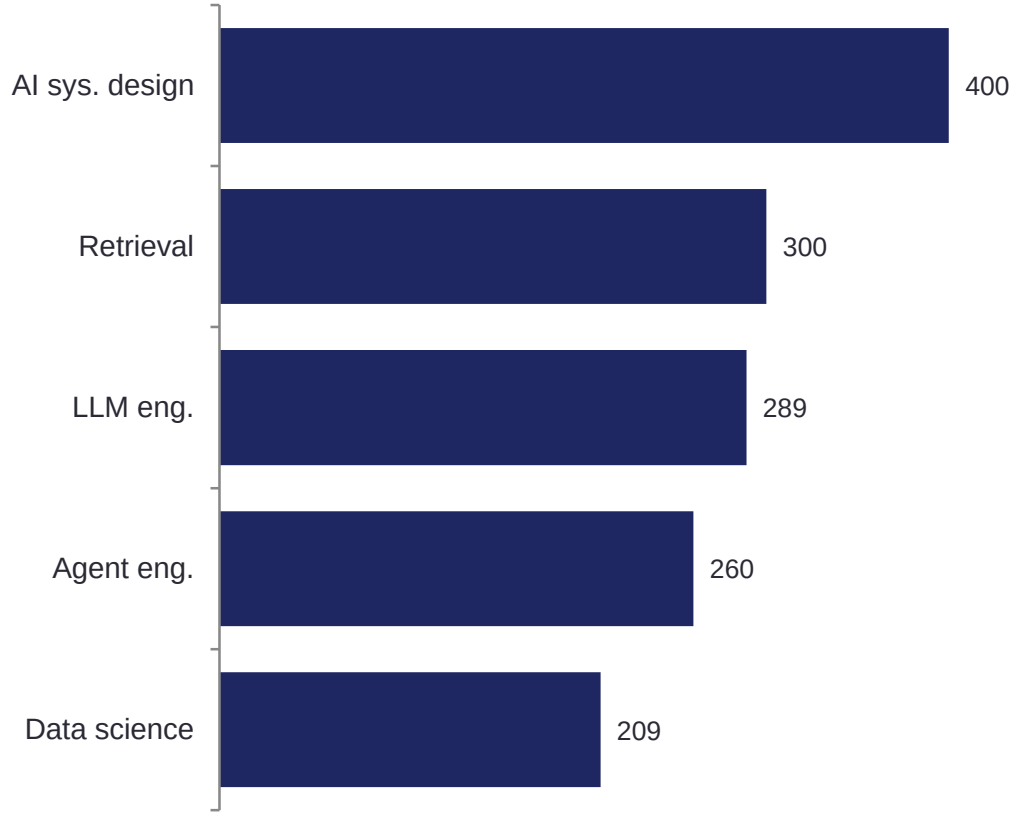
of the 1.562B base model

16

LoRA rank (r) used

Five separate LoRA experts were trained on the same frozen base model — one per technical domain — so each specializes without touching the others' weights.

Training data per domain expert



MoE: routing, not brute force

A router scores how well each expert fits the query, then only the top-k experts contribute to the answer — not all five, every time.

Routing score:

$$s_k(q) = \text{cosine}(f\theta(e(q)), \mu_k)$$

Top-k weights:

$$w_k = \exp(s_k) / \Sigma_j \exp(s_j), \quad j \in \text{top-k}$$

q = query embedding • μ_k = mean latent representation of domain k • top-2 composition used at inference

Weighted merge at inference:

$$W = W_0 + w_1 \cdot \Delta W_1 + w_2 \cdot \Delta W_2$$

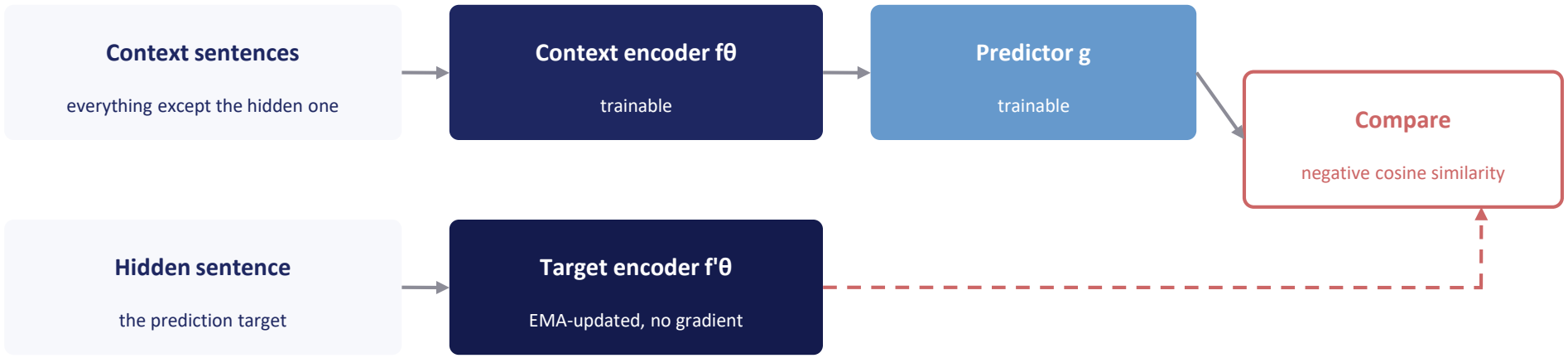
This is a weight-space blend of two adapters, applied in a single forward pass — a design choice this study later stress-tests.

Why not conventional sparse MoE?

Token-level sparse MoE (Switch Transformer, Mixtral) needs pretraining-scale data and compute. This system approximates the idea using adapter-level routing over independently trained LoRA experts — far cheaper, but mechanistically different.

JEPA: predicting meaning, not words

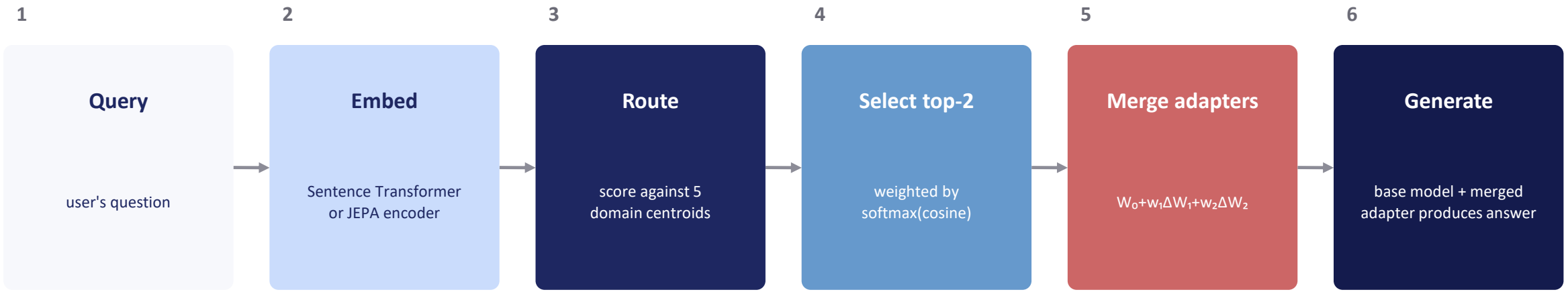
Hide a sentence, guess its meaning (not its exact words) from the sentences around it — repeated thousands of times, this teaches a sense of semantic similarity.



$$L_{JEPA} = L_{pred} + \lambda \cdot L_{var} \qquad L_{pred} = -\cos(\hat{z}_t, z_t) \qquad L_{var} = \text{mean}(\max(0, 1 - \text{std}(z_c)))$$

The variance term (L_var) exists specifically to stop the encoder from collapsing to a trivial constant output — a known failure mode of predictive latent objectives without it.

The full pipeline, end to end



Two things make this different from a conventional MoE Transformer:

Adapter-level, not token-level. Routing happens once per query, selecting whole LoRA experts — not per-token, per-layer routing inside a sparse Transformer block.
Weight-space merge, not output-space mixture. Experts are blended by combining their weight deltas before one forward pass, not by combining two experts' output distributions.

A naming note, upfront

This system is an adapter-level mixture-of-LoRA architecture. It is not a conventional token-level sparse Transformer MoE such as Switch Transformer or Mixtral.

Throughout this deck: “JEPA-inspired routing,” “domain-specialized LoRA experts,” and “adapter-level expert routing” refer to the present implementation. A true sparse Transformer MoE is treated as future work (slide 22).

Dataset and experimental design

Five technical domains, cleaned from a larger raw instruction corpus

Domain	Train examples	Eval examples
AI system design	400	3
Retrieval systems	300	4
LLM engineering	289	6
Agent engineering	260	3
Data science	209	4
Total	**1,458**	**20**

Four systems compared, identical prompts and evaluation set:

Raw base model
unmodified Qwen2.5-Coder-1.5B

Combined fine-tune
one LoRA on all domains pooled

JEPA-routed MoE-LoRA
the proposed system

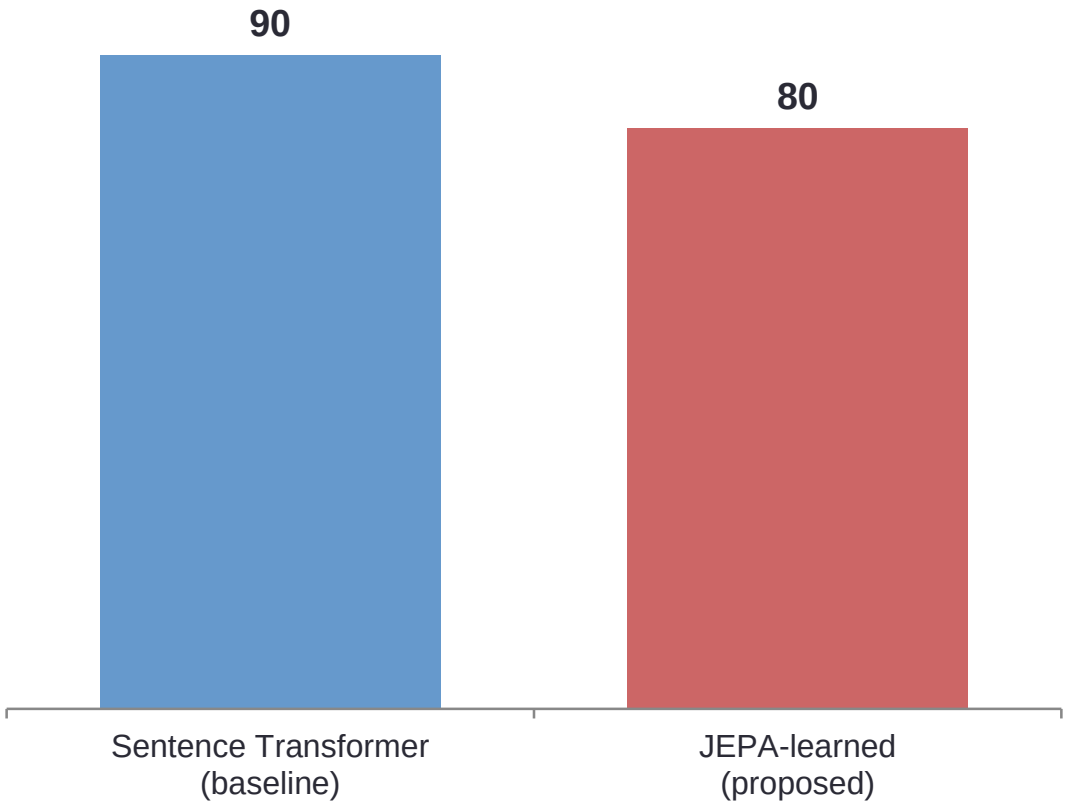
Gemini 1.5 Flash
external reference point

04

Results

Routing accuracy and generation quality — all numbers from real experiments on real data.

Does JEPA route better than a simple baseline?



18/20

baseline correct

16/20

JEPA correct

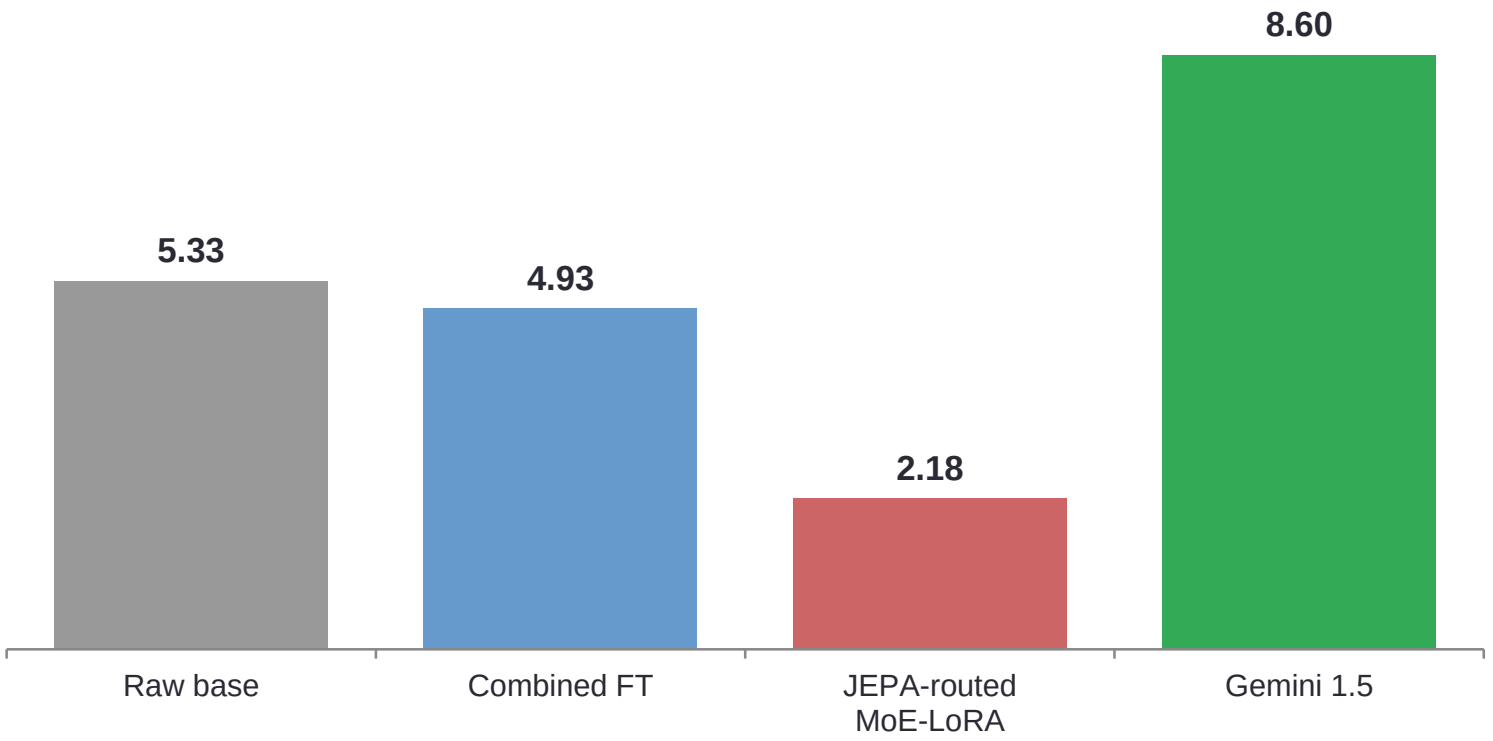
The self-supervised JEPA objective did not outperform the simple baseline on this evaluation set.

Absolute gap: −10 percentage points • Relative gap: −11.1%
This is reported as a genuine negative result — H1 is not supported by this experiment.

It does NOT show JEPA is unsuitable for routing in general — larger datasets, supervised routing losses, and hidden-state representations remain untested (see Roadmap).

Generation quality: the more consequential result

All 20 examples scored on correctness, relevance, and completeness (1-10). The Gemini judge's API quota ran out before scoring, so Claude scored these on the same rubric (disclosed substitution).



KEY FINDING

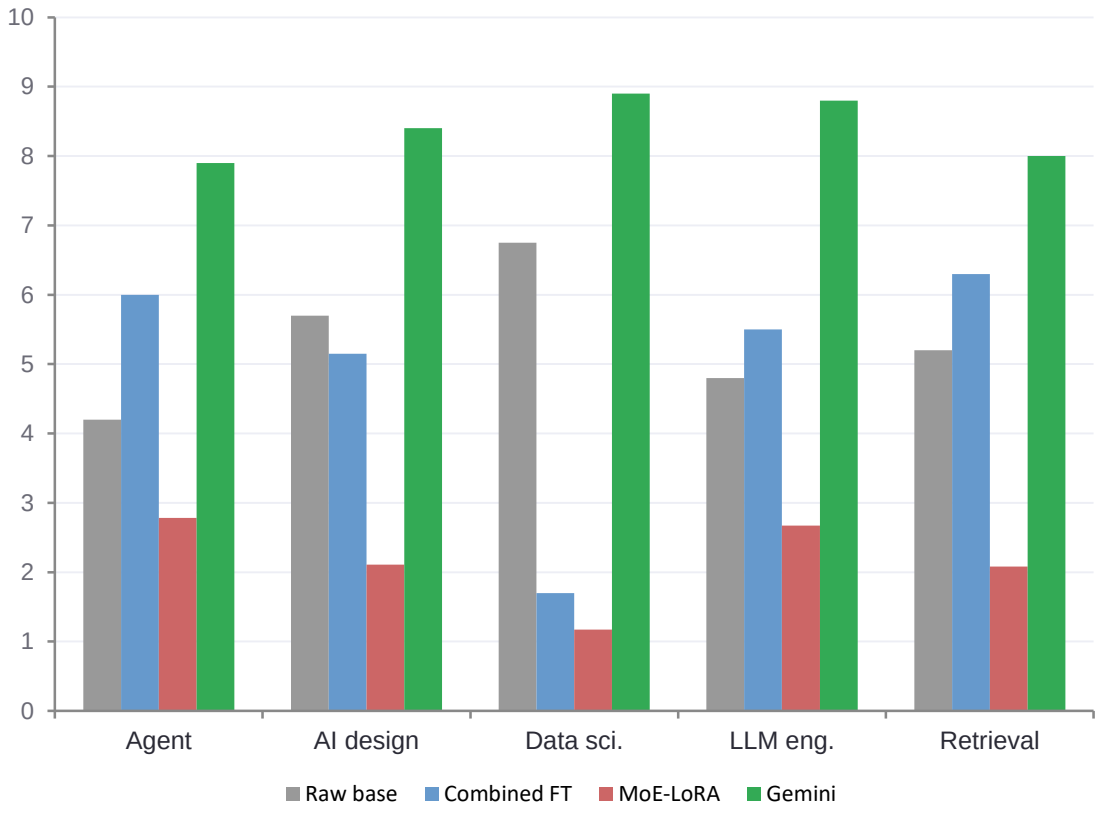
On the same base model, fine-tuning made answers worse, not better.

Raw base > Combined fine-tune > JEPA-routed MoE-LoRA

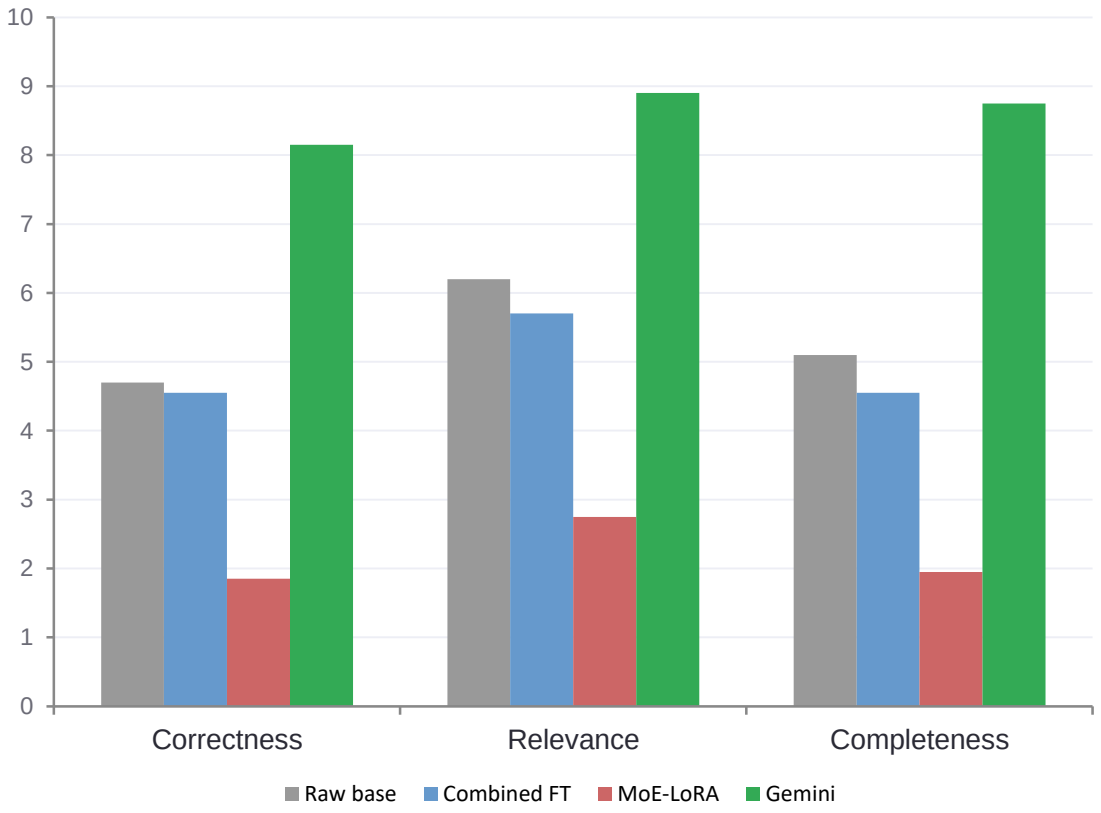
Gemini is a much larger commercial model shown as a reference point, not a same-class comparison.

Breaking the result down

By domain



By metric



What the low scores actually look like

Low scores weren't vague or garbled answers — several were fluent, complete answers to a different question entirely.

PROMPT ASKED

“Explain the concept of Output in data science.”

MOE-LORA RESPONDED WITH

A complete, syntactically correct protein-function-prediction library description — reproduced identically for two different evaluation examples asking the same question.

PROMPT ASKED

“Explain Python in the context of data science.”

MOE-LORA RESPONDED WITH

Opens by literally restating a completely different prompt's instruction text, verbatim, before continuing into unrelated web-scraper code.

05

Why did it underperform?

Three specific, checkable mechanisms — not one vague explanation.

Mechanism 1: weight-space adapter interference

1

The system merges two experts by combining their weight deltas before a single forward pass — not by combining two experts' output distributions.

What this system does:

$$f(x;\ W_0 + w_1\Delta W_1 + w_2\Delta W_2)$$

one forward pass through blended weights

A "true" output-space mixture:

$$w_1 f(x; W_0+\Delta W_1) + w_2 f(x; W_0+\Delta W_2)$$

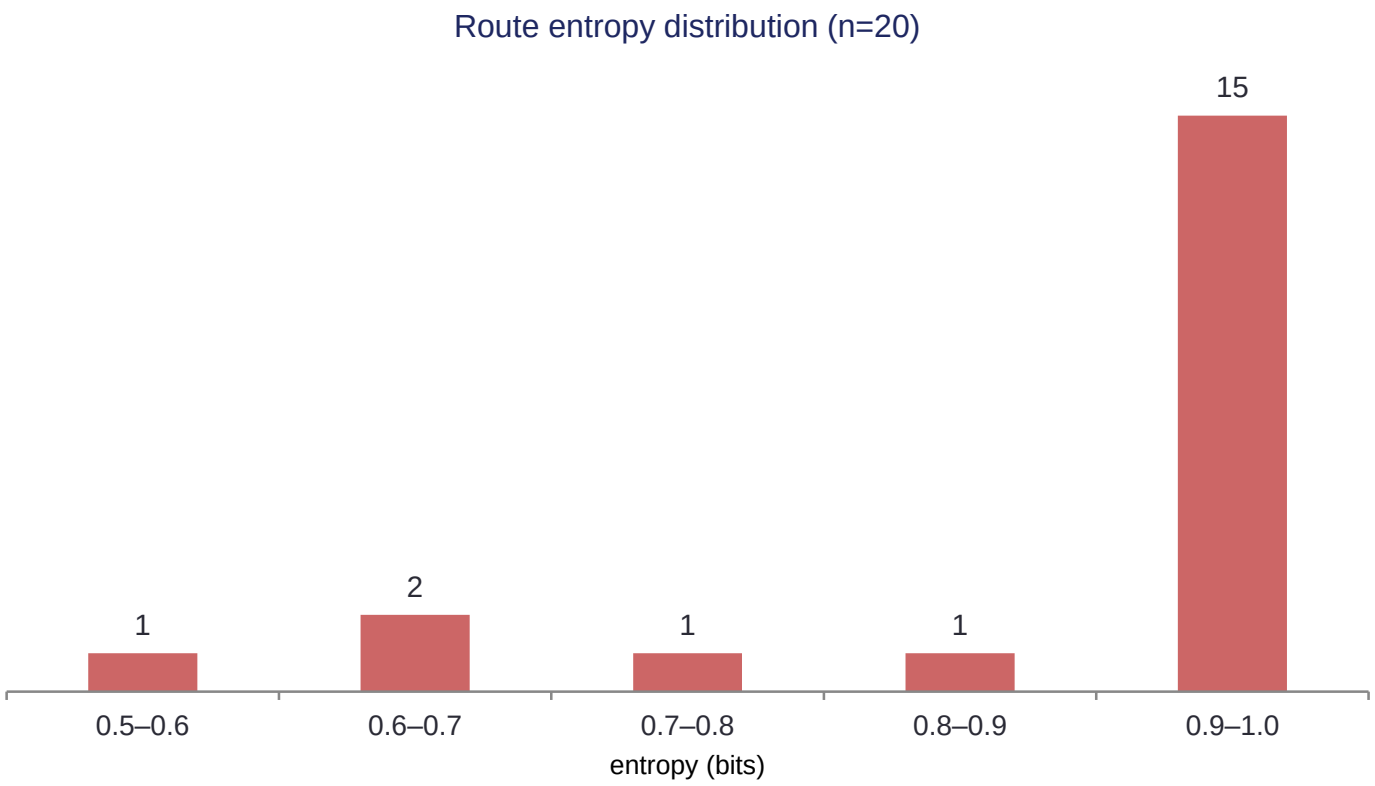
two forward passes, blend the outputs

These are equivalent only if the model is linear in its weights — and a deep Transformer is not. Weight-space averaging can preserve behavior under favorable conditions ("model soups"), but that literature reports carefully-selected checkpoint combinations, not arbitrary similarity-weighted pairs.

Evidence this matters: routing is almost always a near-50/50 blend (next slide) — meaning nearly every generation goes through this untested merge, not a validated single expert.

Evidence: the router rarely commits

Route confidence measured as entropy: $H(w) = -\Sigma\, w_i \cdot \log_2(w_i)$ — 1.0 bit means a perfect 50/50 tie, 0 bits means full confidence in one expert.



0.994

mean route entropy, out of a maximum possible 1.0 bit

10 / 20

examples within 0.001 of an exact tie

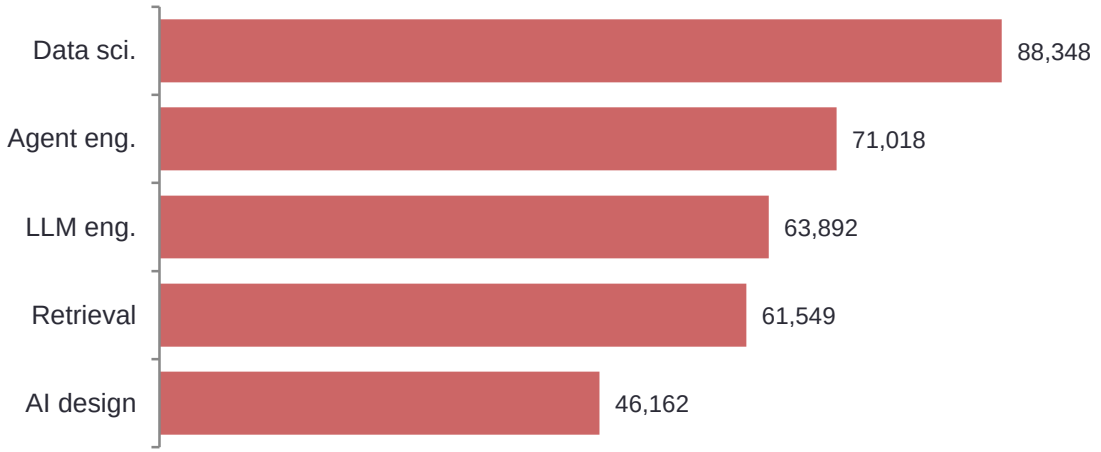
0.005

median gap between the top two route weights

Mechanisms 2 and 3: capacity vs. data

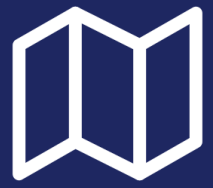
2 Capacity relative to data volume

18.46M trainable params per expert, trained on 209-400 examples in as few as 81 optimizer steps. High params-per-example ratio + very few steps over stylistically templated docs → memorization or "shortcut learning" rather than generalized task understanding.



3 Corpus size correlates with quality

Weak but directionally consistent: domains with more training data scored higher on generation quality (Pearson r = 0.31, n=5 domains — not statistically reliable alone, but consistent with mechanism 2.

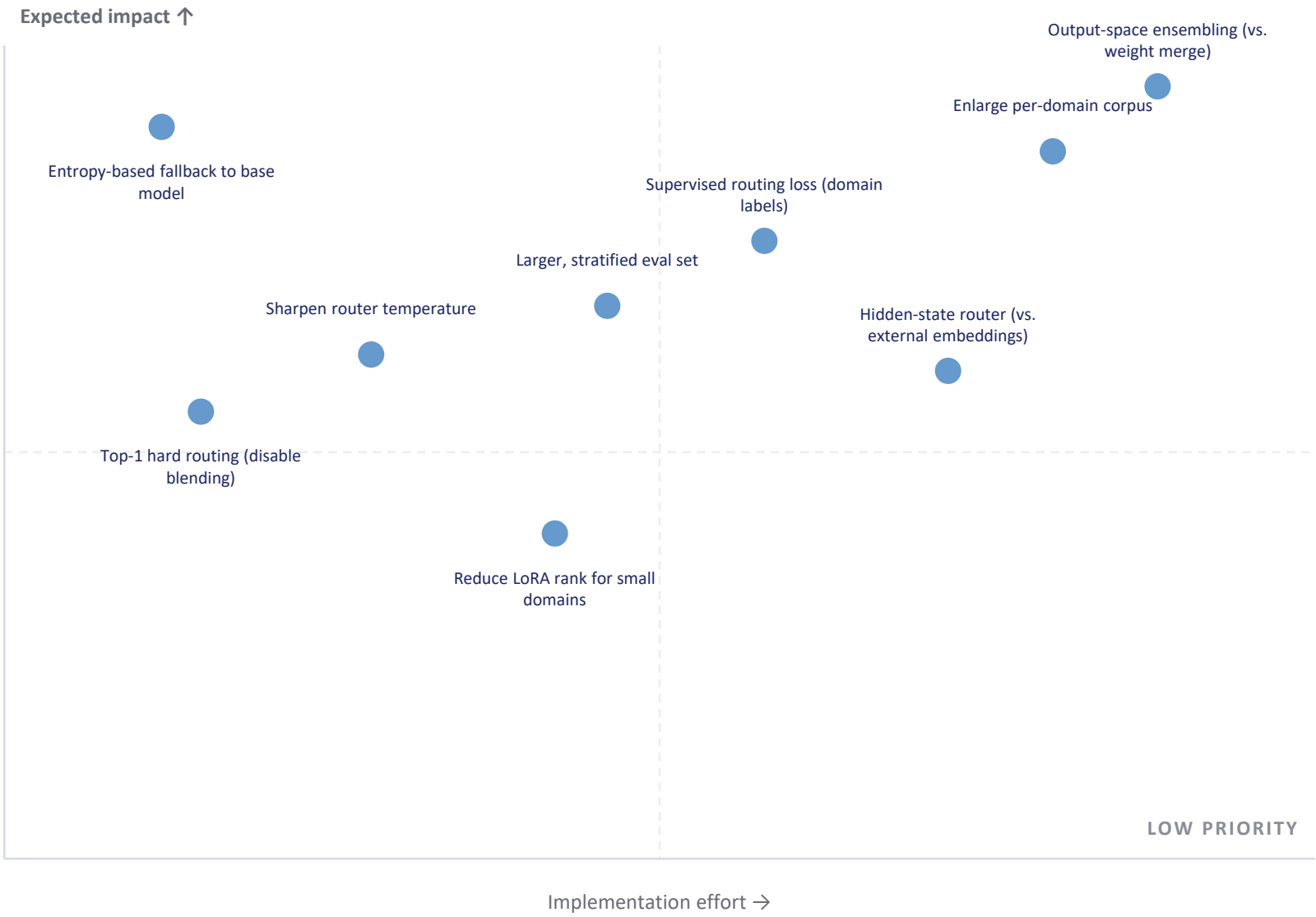


06

Making it usable

Nine concrete fixes, each tied to one of the three diagnosed mechanisms.

Prioritized by impact vs. effort



DO FIRST

Low effort, high impact

Fall back to the raw base model whenever routing confidence is low.

Since the base model already beat both fine-tuned variants (slide 13), this alone should recover baseline quality on ambiguous queries while still using experts where the router is confident.

Proposed as the very first experiment to run — ahead of any higher-effort architecture change.

All nine fixes, in detail

Fix	Targets	Effort
Entropy-based fallback to base model	Weight-space interference	Low
Top-1 hard routing (disable blending)	Weight-space interference	Low
Sharpen router temperature	Router confidence saturation	Low-Med
Reduce LoRA rank for small domains	Capacity/data mismatch	Medium
Supervised routing loss (domain labels)	Router accuracy	Medium
Larger, stratified evaluation set	Statistical reliability	Medium
Enlarge per-domain training corpus	Capacity/data mismatch	Med-High
Hidden-state router (vs. external embeddings)	Router accuracy & alignment	High
Output-space ensembling (vs. weight merge)	Weight-space interference	High

CONCLUSION

What this study actually shows

What worked

- Full pipeline built and runs end to end
- JEPA encoder trained without representational collapse
- Honest, disclosed generation-quality scoring completed

What didn't

- JEPA routing did not beat a simple baseline (80% vs 90%)
- Fine-tuning reduced answer quality vs. the raw base model
- Weighted adapter blending rarely differentiates experts

What's next

- Entropy-based fallback to base model (highest priority)
- Re-test JEPA against real semantic embeddings, more data
- Move from weight-space merge to output-space ensembling

A negative result is still a result.

Thank you — questions welcome.

Elias Tadele